

Appunti — Modulo 3A: Esecuzione e Monitoraggio dei Servizi

Corso: Lab Amministrazione di Sistemi T

Sessione pratica: 2026-04-23 / 2026-04-24

Stato: □ Completato — Es. 1-7 eseguiti sulla VM

1. Demoni (Daemon)

I **demoni** sono processi di sistema che: - si avviano e arrestano automaticamente - non sono collegati a nessun terminale - girano a nome di un utente specifico (esistono UID dedicati ai demoni), senza shell interattiva

“senza shell interattiva” significa? Il demone non ha un terminale da cui leggere input. Non può essere interrogato dall'utente direttamente — gira silenzioso in background. Il suo I/O va verso file di log, socket di rete, o altri meccanismi non interattivi.

Le operazioni tipiche di un amministratore sui demoni: - configurarne avvio e arresto automatico - verificarne lo stato - tracciarne l'esecuzione (log) - avviarli, arrestarli o inviare segnali manualmente

2. Le tre fonti primarie di processi

Oltre agli utenti interattivi, i processi su Linux nascono da tre fonti:

1. **Procedure di avvio del sistema** → gestite dal sistema di init (systemd)
2. **Pianificatori periodici e sporadici** → cron, at, timer systemd
3. **Demoni di gestione degli eventi** → udev (hardware), D-Bus (comunicazione tra processi)

Risposta alle domande: Sì, esatto su tutti i fronti. - Le *procedure di avvio* sono analoghe ai “programmi all'avvio” di Windows — servizi che vengono avviati automaticamente quando il sistema si accende (ssh, cron, rsyslog, ecc.). Qui però hai controllo fine su ogni singolo servizio. - I *pianificatori periodici* sono entrambe le cose: cron gestisce esecuzioni periodiche (ogni ora, ogni giorno, ogni lunedì), at gestisce esecuzioni una-tantum in un istante futuro preciso. - I *demoni di gestione degli eventi* reagiscono a stimoli esterni: udev risponde al collegamento di una USB, D-Bus gestisce la comunicazione inter-processo.

3. Evoluzione del sistema di init

Il processo con **PID = 1** si chiama **init**. È il primo processo avviato dal kernel ed è responsabile — direttamente o indirettamente — di tutti gli altri.

“direttamente o indirettamente” significa? Direttamente: **init** avvia i servizi principali (**sshd**, **cron**, **rsyslog**). Indirettamente: quei servizi, una volta avviati, possono a loro volta creare processi figli. Tutti i processi del sistema discendono da **PID=1**.

SysVinit (storico, anni '80-2000)

- Configurato da **/etc/inittab**
- Avvio **sequenziale** (uno alla volta) — lento
- Gestisce **runlevel** (stati del sistema: 0=spegnimento, 3=multiutente, 5=grafico)
- Script dei servizi in **/etc/init.d/**

Upstart (Ubuntu 6.10-14.10, ora defunto)

- Avvio basato su **eventi** — avvio parallelo, più veloce
- Ogni task risponde a eventi (avvio hardware, rete disponibile, ecc.)

Systemd (attuale standard)

- Avvio **parallelo condizionato** — ogni unit parte appena i vincoli di dipendenza sono soddisfatti
- Sostituisce **init**, **udev**, **cron**, **syslog** e molto altro
- Usato da Debian 8+, Ubuntu 15.04+, praticamente tutto il mondo Linux moderno

SysVinit, Upstart e Systemd sono tutti init? Sì, esatto. Sono tre implementazioni diverse del processo **init** (**PID=1**), apparse in sequenza storica. Oggi si usa **Systemd**. Il kernel avvia sempre e solo **PID=1** — il resto dipende da chi è installato come **init**.

4. Systemd — Concetti fondamentali

4.1 Unit

Systemd gestisce il sistema tramite **unit** — file di testo con nome nella forma **nome.tipo**.

Tipo	Funzione
.service	controllo e monitoraggio di un demone
.socket	attivazione via canale IPC (porta di rete, socket UNIX)
.target	gruppo di unit — sostituisce il concetto di runlevel
.timer	attività legate al tempo (sostituisce cron)
.device	punti di accesso a dispositivi hardware
.mount / .automount	punti di montaggio filesystem
.slice	gestione risorse via cgroup

4.2 Dove trovare le unit

`/lib/systemd/system/` ← libreria di riferimento (pacchetti software)
`/usr/lib/systemd/system/` ← stessa cosa su alcune distro
`/etc/systemd/system/` ← personalizzazioni locali (priorità massima)

Come accedervi? Con i comandi normali: `ls /lib/systemd/system/` per vedere cosa c'è, `cat /lib/systemd/system/ssh.service` per leggere la configurazione di un servizio specifico. Le personalizzazioni in `/etc/systemd/system/` sovrascrivono quelle di sistema.

5. systemctl — Controllo dei servizi

5.1 Operazioni a runtime (volatili)

```

systemctl start    nomeservizio    # avvia
systemctl stop     nomeservizio    # ferma
systemctl restart  nomeservizio    # stop + start
systemctl reload   nomeservizio    # ricarica configurazione senza restart
systemctl status   nomeservizio    # mostra stato, PID, log recenti
  
```

“a runtime” significa? Le modifiche entrano in vigore subito, senza reboot. L'opposto di “persistente”: se riavvii il sistema, queste operazioni non lasciano traccia — il servizio torna allo stato precedente.

`systemctl status` è uno strumento di diagnostica potente: mostra stato corrente, PID, albero processi, risorse (CPU, cgroup), ultimi messaggi di log da `journald`.

5.2 Configurazione persistente (effetto al prossimo boot)

```

systemctl enable  nomeservizio    # avvio automatico al boot
systemctl disable nomeservizio    # disabilita avvio automatico (ma manuale rim
  
```

```
systemctl mask      nomeservizio    # neutralizza completamente (impedisce anche i
systemctl unmask    nomeservizio    # ripristina
```

disable ≠ mask: disable rimuove l'avvio automatico ma lascia fare start manualmente. mask crea un symlink verso /dev/null — il servizio diventa completamente inerte, impossibile avviarlo.

5.3 Ispezione della configurazione

```
systemctl list-units          # tutte le unit attive
systemctl list-units -t service # solo i servizi attivi
systemctl list-unit-files -t service # tutti i servizi installati (anche i
systemctl list-unit-files -t service --state=enabled # solo quelli abilitati a
systemctl --state=failed      # unit in stato failed
```

In che contesto servono? Scenari tipici: - Hai appena installato un pacchetto e vuoi verificare se il servizio si è avviato: `systemctl list-units -t service` - Il sistema si comporta in modo strano: `systemctl --state=failed` per vedere quali servizi sono crashati - Vuoi fare un inventario di cosa gira al boot: `systemctl list-unit-files --state=enabled` - In security: enumeri i servizi attivi su una macchina per valutare la superficie d'attacco

5.4 Target — i nuovi runlevel

SysV Runlevel	Systemd Target	Descrizione
0	poweroff.target	Spegnimento
1	rescue.target	Single user mode (solo root, per riparazione)
3	multi-user.target	Multi-utente, senza GUI
5	graphical.target	Multi-utente con GUI
6	reboot.target	Riavvio

```
systemctl get-default          # target di avvio corrente
systemctl set-default multi-user.target # imposta target di default (effetto
systemctl isolate multi-user.target    # passa a quel target SUBITO (senza r
```

A cosa serve nel tangibile? Il target definisce lo “stato operativo” del sistema — quale insieme di servizi deve essere attivo. `graphical.target` include tutto ciò che sta in `multi-user.target` più l'interfaccia grafica. Usi pratici: - Un server non ha bisogno di GUI: imposti `multi-user.target` come default per risparmiare risorse - `rescue.target` è usato per riparare un sistema che non si avvia correttamente - Il cambio di default NON richiede reboot per l'effetto immediato (`isolate`), ma il `set-default` agisce sul prossimo boot

5.5 Contesto utente (--user)

```
systemctl --user start    mioservizio
systemctl --user enable  mioservizio    # avvio automatico al login (non al boot)
```

Come si relaziona al service manager principale? Sono due istanze completamente separate. Il service manager --user: - Gira nella sessione dell'utente — si avvia al login e termina al logout - Non ha accesso ai servizi di sistema - Non può bindare porte < 1024 - Non richiede root - Viene usato per servizi personali: script di sincronizzazione, ambienti di sviluppo, bot locali

Quello che vedi con systemctl --user list-units è completamente diverso da quello di systemctl list-units.

6. Struttura di un file unit

I file unit sono la **configurazione che dice a systemd come gestire un servizio**. systemd li legge all'avvio e quando esegui systemctl start/enable. Senza un file unit, systemd non sa come avviare un programma — è il “certificato di nascita” di ogni servizio.

Un file unit .service minimo:

```
[Unit]
Description=Il mio servizio di esempio
Requires=network.target
After=network.target

[Service]
Type=exec
ExecStart=/usr/bin/mioprogramma --opzione
ExecStop=/usr/bin/mioprogramma --stop
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

6.1 Sezione [Unit] — dipendenze

B e C nelle direttive sono **placeholder per il nome di un'altra unit**. Esempio concreto: stai scrivendo nginx.service. Vuoi che nginx parta solo dopo che la rete è pronta. Scrivi Requires=network.target e After=network.target — dove network.target è “B”.

Direttiva	Significato
Requires=B	B deve essere attiva; se B fallisce, questa si ferma
Wants=B	come Requires ma tollerante: se B fallisce, questa continua
After=B	avvia <i>dopo</i> B (Wants/Requires non implicano ordine temporale!)
Conflicts=B	mutuamente esclusivi con B
OnFailure=C	avvia C se questa fallisce
WantedBy= (in [Install])	crea automaticamente un Wants nella unit indicata all'enable

Regola critica: Wants=network.target senza After=network.target significa “avvia in parallelo con network.target e tollerami se non è pronto”. Per la sequenza corretta servono entrambi.

6.2 Tipi di avvio (Type= nella sezione [Service])

Type	Comportamento
exec	systemd considera il servizio avviato quando il comando ExecStart è effettivamente in esecuzione (<i>consigliato per nuovi servizi</i>)
simple	come exec ma systemd dichiara il servizio avviato già al fork — meno sicuro
forking	il processo si sdoppia e il genitore esce (demoni Unix classici)
notify	come exec, ma il programma invia esplicitamente segnale di readiness via sd_notify
oneshot	esegue una volta sola e termina; utile per script di configurazione
idle	attende che tutti gli altri job siano smistati, poi parte

Per servizi che configurano qualcosa all'avvio e lo ripristinano allo stop (oneshot):

```
ExecStart=/usr/bin/mio-configuratore
ExecStop=/usr/bin/mio-de-configuratore
```

6.3 Riavvio automatico (Restart=)

```
Restart=on-failure    # riavvia se termina con exit code non zero o segnale non
Restart=always        # riavvia sempre, tranne se fermato da systemctl stop
```

7. Esecuzioni pianificate

7.1 at — esecuzione differita (una tantum)

at pianifica un comando da eseguire **una volta sola** in un momento futuro.

```
sudo apt install at # installa se non presente
```

Pianificare

```
echo "/bin/comando" | at now + 30 minutes
echo "/bin/comando" | at 14:30
echo "/bin/comando" | at midnight 31.12.2025
```

Gestire la coda

```
atq # lista job in coda
atrm <job_id> # rimuove un job
```

Come funziona echo "/bin/echo ciao" | at? echo stampa la stringa /bin/echo ciao sul terminale. La pipe | la passa come input ad at, che la interpreta come un comando da eseguire nel momento indicato. È come consegnare un bigliettino ad at con scritto “esegui questo”. Il comando può essere qualunque cosa: un path a uno script, una sequenza bash, un binario di sistema.

Come ricavo il numero del job? Quando pianifichi con at, stampa su stderr: job 5 at Thu Apr 23 15:30:00 2026. Il numero è nella prima colonna di atq. Oppure catturalo automaticamente:

Pattern per salvare l'ID del job (per cancellarlo da script)

```
ATJOB=$(mktemp)
```

```
echo "/bin/comando" | at now + 30 minutes 2>&1 | grep '^job ' | cut -f2 -d' ' >
```

2>&1 → redirige stderr su stdout per poterlo pipare

grep '^job ' → filtra la riga "job 5 at ..."

cut -f2 -d' ' → prende il secondo campo (il numero)

Per cancellare in seguito:

```
atrm $(cat $ATJOB)
```

L'orario della VM è sballato: è normale se la VM non è sincronizzata con l'host. Si risolve con `sudo timedatectl set-ntp true` oppure `sudo ntpdate pool.ntp.org`. Non impatta il funzionamento di at ma i log avranno timestamp sbagliati.

7.2 cron — esecuzione periodica

crond legge i file crontab ogni minuto e lancia i task pianificati.

```
crontab -l          # mostra il crontab dell'utente corrente
crontab -e          # edita il crontab
crontab FILE        # sostituisce il crontab con FILE
```

Sintassi crontab:

MINUTO ORA G.MESE MESE G.SETTIMANA <comando>

Esempi commentati:

```
30 4 1,15 * 6      /bin/backup
# alle 4:30 | giorno 1 e 15 del mese | qualsiasi mese | ogni sabato (6)
# (nota: giorno-mese OR giorno-settimana quando entrambi specificati)
```

```
*/10 8-18 * * 1-5 /home/las/lavora.sh
# ogni 10 min (0,10,20,30,40,50) | ore 8-18 | qualsiasi giorno | qualsiasi mese | lun-ven
```

```
0 0 1 1 *          /usr/sbin/auguri
# alle 00:00 | giorno 1 | gennaio | qualsiasi giorno della settimana
```

Asterischi in dettaglio (seconda riga: `*/10 8-18 * * 1-5`): `- */10` = ogni 10 minuti — la notazione `/P` significa “ogni P unità”. `*/10` sui minuti = 0, 10, 20, 30, 40, 50 - `8-18` = range orario, dalle 8:00 alle 18:00 incluse - `*` = qualsiasi (primo `*` = qualsiasi giorno del mese, secondo `*` = qualsiasi mese) - `1-5` = lunedì (1) al venerdì (5). 0 e 7 sono domenica. Risultato: lo script gira ogni 10 minuti durante le ore lavorative, solo nei giorni feriali.

Eccezione: se sia G.MESE che G.SETTIMANA sono specificati (entrambi $\neq$ *), vengono considerati in **OR logico**, non AND. `30 4 1,15 * 6` → alle 4:30 del giorno 1, del giorno 15, E di ogni sabato — non “solo i sabato che cadono il 1 o 15”.

Crontab di sistema (`/etc/crontab`): ha un campo aggiuntivo per specificare l'utente:

```
30 4 * * * root /usr/sbin/backup
# Alle 4:30 ogni giorno, eseguito come root
```

A cosa serve il crontab di sistema? È il crontab globale della macchina, gestito dall'amministratore. Serve per task di manutenzione di sistema (backup notturni, pulizia log, report di sistema) che devono girare come utenti specifici. Il crontab utente (`crontab -e`) è personale — quel che pianifichi lì gira come te.

Manipolazione programmatica del crontab (da script, senza aprire l'editor):

```
PERIODO='*/10 8-18 * * *'
```



```

COMANDO='/home/las/qualcosa.sh parametri vari'
TMPTAB=$(mktemp)
crontab -l | grep -vF "$COMANDO" > $TMPTAB    # 1. scarica crontab attuale senza
echo "$PERIODO $COMANDO" >> $TMPTAB           # 2. aggiunge la nuova riga
crontab $TMPTAB                                # 3. ricarica il crontab dal file
rm $TMPTAB                                     # 4. pulizia

```

Perché questo pattern? Se da uno script vuoi aggiungere/rimuovere una riga dal crontab senza aprire l'editor interattivo, devi: scaricare il crontab corrente, modificarlo, ricaricarlo. Il `grep -vF "$COMANDO"` rimuove eventuali righe già presenti con quel comando (evita duplicati se lo script viene eseguito più volte).

7.3 Timer systemd — sostituzione moderna di cron e at

Puoi sostituire sia cron che at con due file unit: un `.service` e un `.timer`.

Timer periodico (equivalente cron):

`/etc/systemd/system/myscript.service:`

```

[Unit]
Description=Il mio script periodico

```

```

[Service]
Type=oneshot
ExecStart=/path/to/script.sh

```

`/etc/systemd/system/myscript.timer:`

```

[Unit]
Description=Timer per myscript

```

```

[Timer]
OnCalendar=*-*-* 04:30:00    # ogni giorno alle 4:30
Persistent=true             # esegue anche se la macchina era spenta

```

```

[Install]
WantedBy=timers.target

```

```

sudo systemctl enable --now myscript.timer    # attiva il timer subito e al boot
systemctl list-timers                          # mostra tutti i timer attivi e l

```

Il timer attiva il `.service` corrispondente allo scadere. L'`enable -now` equivale a `enable + start` in un colpo solo. `Persistent=true` è utile su macchine che possono essere spente: se la macchina era spenta all'orario programmato, eseguirà il task al prossimo avvio.

Timer differito (equivalente at):

```
systemd-run --on-active=30m /path/to/script.sh # tra 30 minuti
systemd-run --on-calendar="2025-12-24 11:00:00" /path/to/script.sh
systemd-run --user --on-active=1m /bin/cmd # come utente normale
```

--on-active cosa misura? Misura il tempo dall'avvio del comando systemd-run, non dall'accensione del sistema. --on-active=30m = "esegui tra 30 minuti da adesso". Non è legato al tempo di uptime del sistema.

Sintassi OnCalendar:

DayOfWeek YYYY-MM-DD HH:MM:SS

Placeholder	Valori
DayOfWeek	Mon, Tue, Wed, Thu, Fri, Sat, Sun (opzionale)
YYYY	anno numerico, es. 2025, oppure * per ogni anno
MM	mese 01-12, oppure *
DD	giorno 01-31, oppure *
HH:MM:SS	ore:minuti:secondi

Operatori: - * = qualsiasi valore - A..B = range (es. Mon..Fri) - A,B,C = lista (es. Mon,Wed,Fri) - /P = ogni P unità (es. */2 nei minuti = ogni 2 minuti)

Esempi:

```
*-*-* 04:30:00 # ogni giorno alle 4:30
Mon..Fri *-*-* 08:00:00 # ogni giorno feriale alle 8
*-*-1 00:00:00 # il primo di ogni mese a mezzanotte
*-*-* */2:00:00 # ogni 2 ore
```

Verifica la prossima scadenza di un'espressione prima di usarla:

```
systemd-analyze calendar "Mon *-*-* 04:00"
```

8. Monitoraggio

8.1 Log di sistema — architettura generale

I log sono indispensabili per diagnostica e rilevamento intrusioni. Vanno protetti: integrity checker, replica su macchine remote fuori dalla rete amministrata.

Architettura: 1. I processi inviano messaggi al logger 2. Il logger serializza, fa timestamping e classifica 3. Il logger smista verso destinazioni configurate (file, server remoto, console)

Ogni messaggio ha una coppia <facility>.<priority>:

Facility (chi ha generato il messaggio):

Facility	Sorgente
auth	autenticazione, login
cron	cron e at
daemon	demoni generici
kern	kernel
mail	sistema di posta
user	applicazioni utente
local0-local7	facility personalizzate (per le tue applicazioni)

Priority (gravità, in ordine decrescente):

Priority	Significato
emerg	sistema inutilizzabile
alert	azione immediata richiesta
crit	condizione critica
err	errore
warning	avvertimento
notice	evento normale ma significativo
info	informazione operativa
debug	informazione di debug dettagliata

Come funziona la soglia? Una regola che specifica err fa match con err, crit, alert ed emerg — tutte le priority uguali o superiori. Prefisso = per exact match: =err cattura solo gli errori, non i critici.

8.2 rsyslog (standard su Debian)

rsyslog è il successore di syslog, installato di default su Debian.

File di configurazione: - Direttive globali: /etc/rsyslog.conf - Regole aggiuntive: /etc/rsyslog.d/*.conf ← qui aggiungi le tue regole

Sintassi di una regola:

facility.priority destinazione

Destinazioni possibili: - /path/al/file → scrive su un file - |/path/al/programma → invia a un programma (pipe) - * → tutti gli utenti connessi - @hostname → server remoto via UDP - @@hostname → server remoto via TCP (più affidabile)

Esempi:

```
kern.*          /dev/console          # tutto del kernel → console
*.info;mail.none /var/log/messages          # tutto info+ tranne mail
*.emerg         *                    # emergenze → tutti gli utenti
kern.crit       |/usr/bin/alerter    # critici del kernel → programma
*.=warning      @loghost             # solo warning → server remoto UDP
```

mail.none significa? È una sintassi speciale: none come priority esclude completamente quella facility. *.info;mail.none = “tutto con priority info o superiore, TRANNE qualsiasi messaggio di mail”.

Aggiungere una facility personalizzata:

```
echo "local4.=info /var/log/myevents.log" > /etc/rsyslog.d/mylog.conf
systemctl restart rsyslog
```

```
# Testa con logger
logger -p local4.info "messaggio di test"
tail /var/log/myevents.log
```

```
# Per disabilitarla:
mv /etc/rsyslog.d/mylog.conf /etc/rsyslog.d/mylog.off
systemctl restart rsyslog
```

8.3 systemd-journald

journald è il sistema di log integrato in systemd.

Relazione tra journald e rsyslog: Sono due sistemi paralleli. journald raccoglie i log di tutti i servizi systemd direttamente (in formato binario, dalla nascita alla morte di ogni processo). rsyslog è il demone tradizionale di logging testuale. Possono coesistere: - journald raccoglie tutto, rapidamente, fin dal boot - Se configurato con ForwardToSyslog=yes in /etc/systemd/journald.conf, passa i log anche a rsyslog - rsyslog li smista in file /var/log/*.log, li filtra per facility/priority, li manda a server remoti

In pratica: journalctl per cercare log in tempo reale e per servizi specifici; rsyslog per log persistenti in formato testo leggibile da altri strumenti.

Caratteristiche di journald: - formato **binario** (non testo) → visualizzabile solo con journalctl - attivo dal boot, non dipende da altri servizi - rotazione automatica per età o dimensione

```
journalctl          # tutto il journal
journalctl -u nomesevizio # log di un servizio specifico
journalctl -f        # segui in tempo reale (come tail -f)
journalctl --since "1 hour ago" # filtra per tempo
```

```
journalctl -p err           # solo priority err e superiori
journalctl -b               # log del boot corrente
journalctl -b -1            # log del boot precedente
```

8.4 Diagnostica istantanea del sistema

□ Questa sezione non era presente negli appunti grezzi — inclusa perché fa parte del modulo e della lezione di riferimento.

Comando	Cosa mostra
ps aux	snapshot di tutti i processi attivi
uptime	tempo di attività + load average (1', 5', 15')
free	occupazione RAM e swap
top	combinazione interattiva di ps+uptime+free+CPU
vmstat 1	memoria, paging, I/O, CPU aggiornato ogni secondo
iostat /dev/sda1	statistiche I/O su un dispositivo
df	spazio disco per filesystem
du -s /directory	spazio occupato da una directory
fuser /path	quali processi usano un file/filesystem
lsof	tutti i file aperti da tutti i processi

Lettura di uptime:

```
21:27:56 up 7:10, 2 users, load average: 0.00, 0.00, 0.00
      └─ tempo attivo └─ utenti └─ 1' └─ 5' └─ 15'
```

Il load average va interpretato in rapporto al numero di CPU: load=2 su 4 CPU = sistema al 50% di carico.

Lettura di free:

```

                total    used    free   shared  buff/cache   available
Mem:           237040  121248    9596    1464     106196     98720
```

available $\approx$ free + buff/cache — la cache può essere liberata se serve. La colonna rilevante è available, non free.

9. Esercizi del Lab — Sessione pratica

Es. 1 — Installare at e pianificare un comando □ Eseguito

```
# Installa at
sudo apt install at
```

```
# Pianifica un comando tra 30 minuti
```

```

echo "/bin/echo ciao" | at now + 30 minutes

# Verifica la coda dei job
atq

# Cancella il job (N = numero del job da atq)
atrm N

```

Es. 2 — Watchdog con at □ Eseguito

Pattern: limita il tempo di esecuzione di un processo. Se supera il limite, viene killato.

```

COMANDO_LUNGO &          # avvia in background
PID=$!                   # cattura il PID del processo appena avviato
WD=$(mktemp)
echo "/bin/kill $PID" | at now + $LIMITE minutes 2>&1 | grep '^job ' | cut -f2
# Al completamento normale del processo, cancella il watchdog:
atrm $(cat $WD)

```

Es. 3 — Nice execution (script con at) □ Eseguito

Logica: se il load del sistema è sotto soglia, esegui il comando; altrimenti rischedula tra 2 minuti.

```

THIS=/home/vagrant/niceexec-base.sh
LOAD=$(uptime | awk -F 'average: ' '{ print $2 }' | cut -f1 -d$(locale decimal_
if test $LOAD -lt "$1" ; then
    shift
    eval "$@"
else
    echo $THIS "$@" | at now +2 minutes
fi

```

Es. 4 — Manipolazione crontab da script □ Eseguito

```

PERIODO='*/10 8-18 * * *'
COMANDO='/home/las/qualcosa.sh parametri vari'
TMPTAB=$(mktemp)
crontab -l | grep -vF "$COMANDO" > $TMPTAB
echo "$PERIODO $COMANDO" >> $TMPTAB
crontab $TMPTAB
rm $TMPTAB

```

Es. 5 — Configurazione rsyslog □ Eseguito

```
echo "local4.=info /var/log/myevents.log" | sudo tee /etc/rsyslog.d/mylog.conf
sudo systemctl restart rsyslog
```

```
logger -p local4.info "messaggio di test"
tail /var/log/myevents.log
```

Es. 6 — Script logconfig □ Eseguito

Script che prende facility.priority e un percorso file, configura rsyslog per loggare quella facility sul file dato, e aggiunge a cron un task che ogni 5 minuti manda l'output di uptime a syslog.

```
#!/bin/bash
T=$(mktemp)
echo -e "$1\t$2" > $T
sudo cp $T /etc/rsyslog.d/logconfig.conf
sudo systemctl restart rsyslog.service

CRONCOMMAND="/usr/bin/uptime | /usr/bin/logger -p $1"
crontab -l | grep -Fv "$CRONCOMMAND" > $T
echo "*/* * * * * $CRONCOMMAND" >> $T
crontab $T
rm -f $T
```

Es. 7 — Esaminare unit systemd reali □ Eseguito

```
# Unit oneshot (configura qualcosa e poi ripristina)
cat /lib/systemd/system/networking.service

# Unit forking (demone in background)
cat /lib/systemd/system/dnsmasq.service

# Unit con Restart (riavvio automatico)
cat /lib/systemd/system/cron.service
cat /lib/systemd/system/rsyslog.service

# Controllare log di una unit
journalctl -u ssh.service
journalctl -u cron.service -f
```

Cheat sheet systemctl

Operazione	Comando
Avvia ora	<code>systemctl start nome</code>
Ferma ora	<code>systemctl stop nome</code>
Stato + log	<code>systemctl status nome</code>
Riavvia	<code>systemctl restart nome</code>
Ricarica config	<code>systemctl reload nome</code>
Abilita al boot	<code>systemctl enable nome</code>
Disabilita al boot	<code>systemctl disable nome</code>
Blocca completamente	<code>systemctl mask nome</code>
Lista servizi attivi	<code>systemctl list-units -t service</code>
Lista tutti i servizi	<code>systemctl list-unit-files -t service</code>
Solo i falliti	<code>systemctl --state=failed</code>
Target corrente	<code>systemctl get-default</code>
Log di un servizio	<code>journalctl -u nome</code>
Log in tempo reale	<code>journalctl -u nome -f</code>